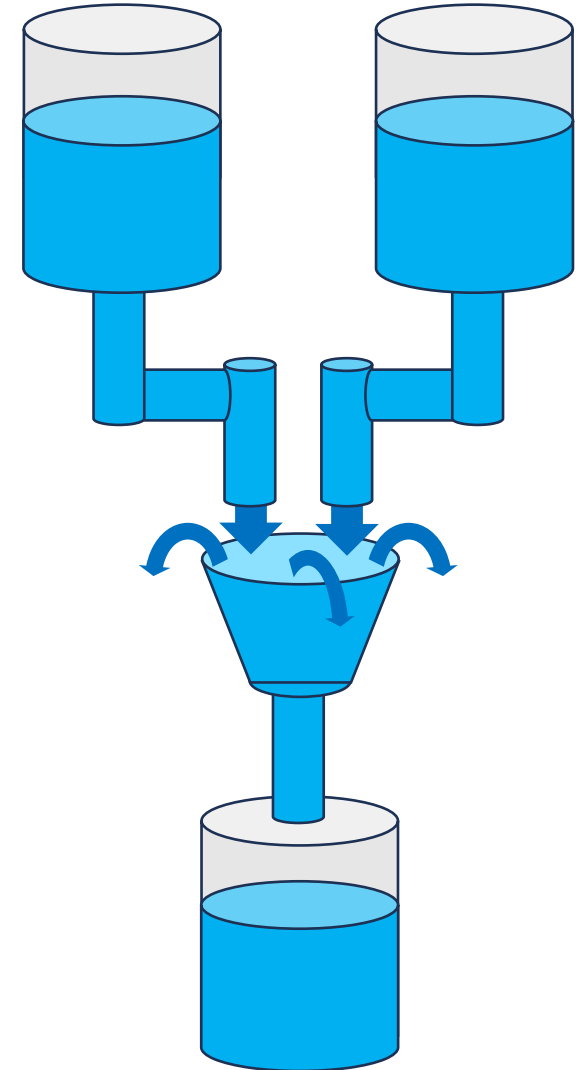


Transport Layer

Congestion and Flow Control Problem

- Additional features of TCP with respect to UDP are the **congestion and flow control**.
- We mentioned that packet loss is **often a result of buffers' overflow**:
 - From receiver buffer, if **receiving application is not fast enough** in reading the data.
 - From network devices (e.g., routers), if **nodes are congested**.
- Following our previous water-flowing analogy, **we can see buffers as intermediate buckets** that overflows in case water exceeds.
- If packets are lost, **hosts are forced to rely on retransmission and timeouts** that drastically impair the network performance.



Transport Layer

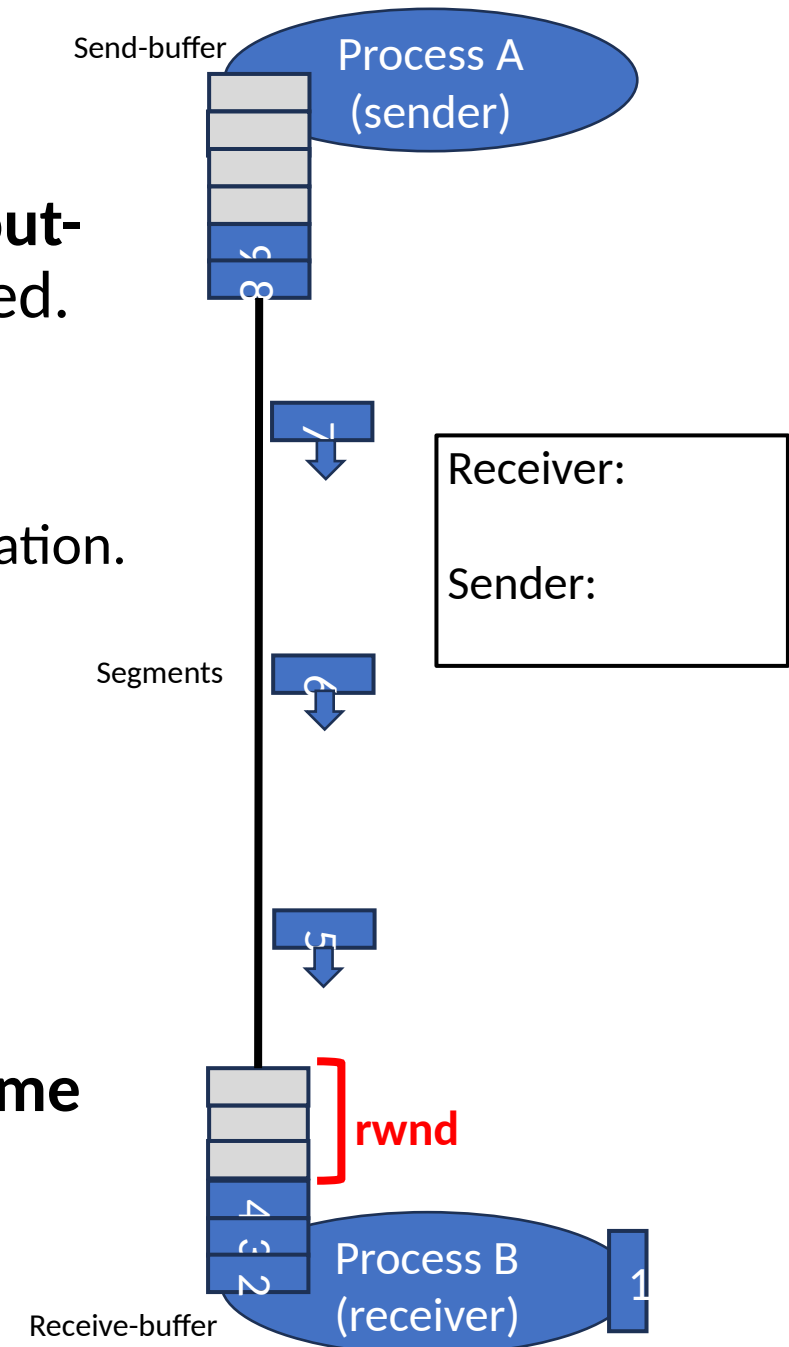
Flow Control

- When the TCP connection receives bytes that are correct and in sequence, it places the data in the receive buffer. The receiving application **will read data from this buffer**, but **not necessarily at the instant the data arrive** (application may be busy).
 - If the application is relatively **slow at reading the data**, the sender can very easily **overflow the receiver buffer** by sending too much data too quickly.
- **TCP flow control is a speed-matching service**: it attempts to match (reduce) the rate at which the sender is sending against the rate at which the receiving application is reading.
- In this case we rely on a **receive window** to inform the sender how many segments we can receive without producing overflows.
 - Remind: TCP segment **allow receiver to tell the sender** how much buffer has left through the **receive window field** (free buffer space).

Transport Layer

Flow Control

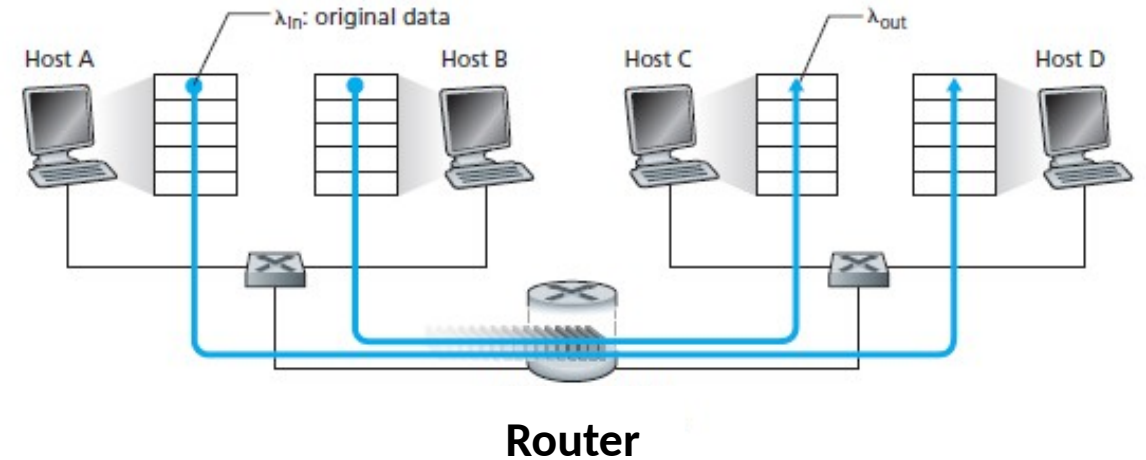
- Let's assume for simplicity that the **TCP receiver discards out-of-order segments**, so all segments in the buffer are ordered.
- On the **receiver side** (host B) we have:
 - **RcvBuffer**: size of the receiver buffer (in bytes)
 - **LastByteRead**: the last byte of the stream retrieved by the application.
 - **LastByteRcvd**: the last byte of the stream that is received.
 - We may define the size of the **receive window** (**rwnd**) as:
$$rwnd = RcvBuffer - (LastByteRcvd - LastByteRead)$$
- On the **sender side** (host A) we have:
 - the last byte of the stream to be sent.
 - the last byte of the stream that is acknowledged by the receiver.
- Knowing the receive window, the **sender guarantees anytime** that:
$$LastByteSent - LastByteAckd \leq rwnd$$



Transport Layer

Congestion Problem

- The **congestion control problem** is similar to flow control, but it is related to the network infrastructure.
- Example: two hosts (A and B) have a connection that **shares a single router** and a single outgoing link of capacity between source and destination.



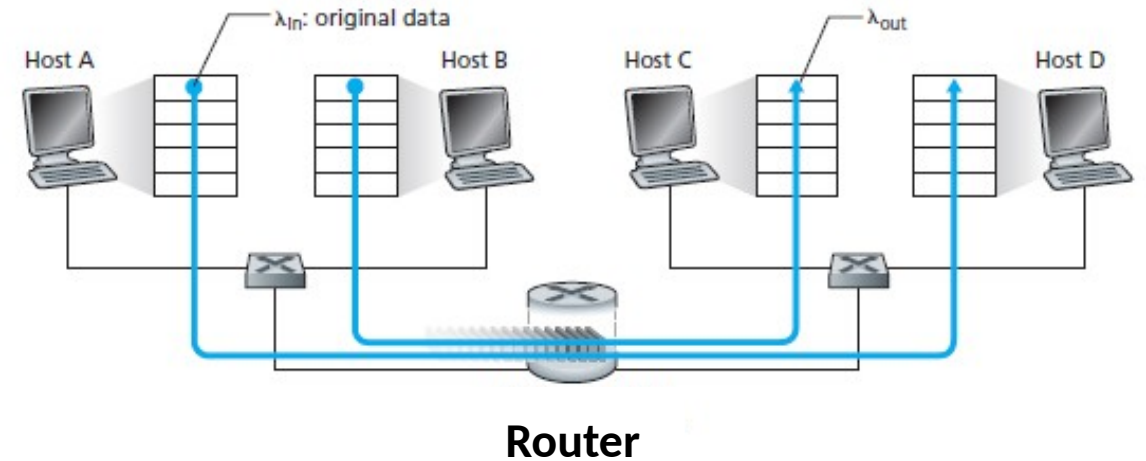
- The **router has buffers** that allow it to store incoming packets when the packet-arrival rate exceeds the outgoing link's capacity.
- **Link capacity:** R

Let's assume for simplicity that both applications (in A and B) are sending data into the connection at **the same average rate** of λ_{in} bytes/s.

Transport Layer

Congestion Problem – simplified view

- If $\lambda_{in} < R/2$, everything sent by the sender is received by the receiver **with a finite delay**.
- If $\lambda_{in} = R/2$, **the link reaches its full capacity** (R), and the exceeding packets are stored into the buffer.



- As long as we exceed the maximum capacity the **packets will be accumulating into the router's buffer** waiting for their turn to be sent into the link.
- Since the **buffer is finite**, accumulated **packets will eventually be discarded**, and hosts will be forced to retransmit them (even more traffic).
- **An infinite buffer doesn't work either**: packets will not be lost but **the delay would constantly increase**: if we exceed the capacity forever, the delay will reach infinity (so packets are as good as lost).

Transport Layer

Congestion Control

- There are **mainly two approaches** to congestion control:
 1. **End-to-end congestion control**: this is the standard TCP approach where **congestion is inferred by the end systems** based only on observed network behavior (for example, packet loss and delay).
 - TCP segment loss (due to timeouts or the receipt of 3 duplicate ACKs) is taken as an indication of network congestion, and TCP decreases sending rate accordingly.
 2. **Network-assisted congestion control**: this is a recent (optional) approach where transport-layer works in synergy with network-layer. **Routers provide explicit feedback** to the sender and/or receiver regarding the congestion state of the network.
 - The CWR and ECE flags are used to this end.
 - The feedback may be as simple as a **single bit indicating congestion** at a link, but more sophisticated feedback is also possible.

Transport Layer

TCP Congestion Control

- **TCP mostly relies on end-to-end congestion control.**
- The approach is to have **each sender to adapt their sending rate** depending on the **perceived network congestion**.
- Here there are 3 main problems to be considered:
 - **Rate regulation:** how does a TCP sender regulate the rate at which it sends traffic into its connection?
 - **Congestion detection:** how does a TCP sender detect the congestion on the path between itself and the destination?
 - **Rate adjustment:** what algorithm should the sender use to change its send rate as a function of perceived end-to-end congestion?

Transport Layer

TCP Congestion Control – Rate Regulation

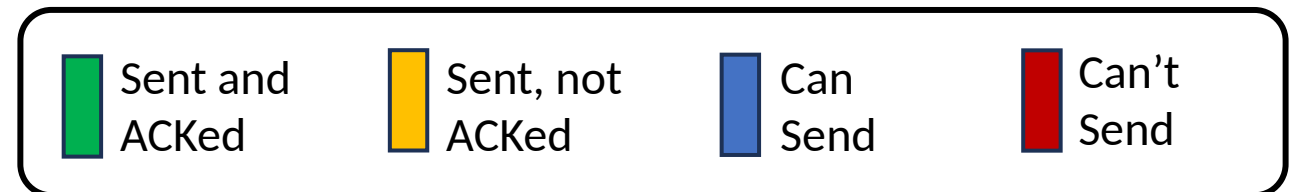
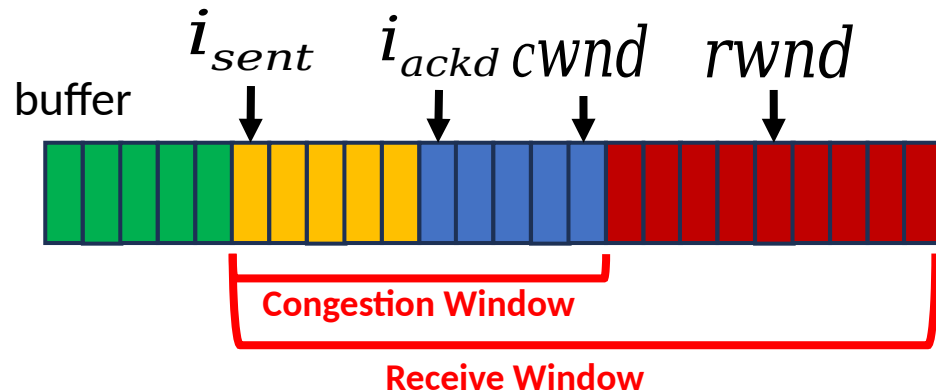
- Reminder: in TCP flow control **the sending rate is regulated by considering the buffer space on the receiver side** (receiver window, *rwnd*):

$$\text{LastByteSend} - \text{LastByteAcked} \leq \text{rwnd}$$

- In TCP congestion-control the sender **also keeps track of a congestion window** (*cwnd*):

$$\text{LastByteSent} - \text{LastByteAcked} \leq \min\{\text{cwnd}, \text{rwnd}\}$$

- So, the rate is regulated by increasing/decreasing .



Transport Layer

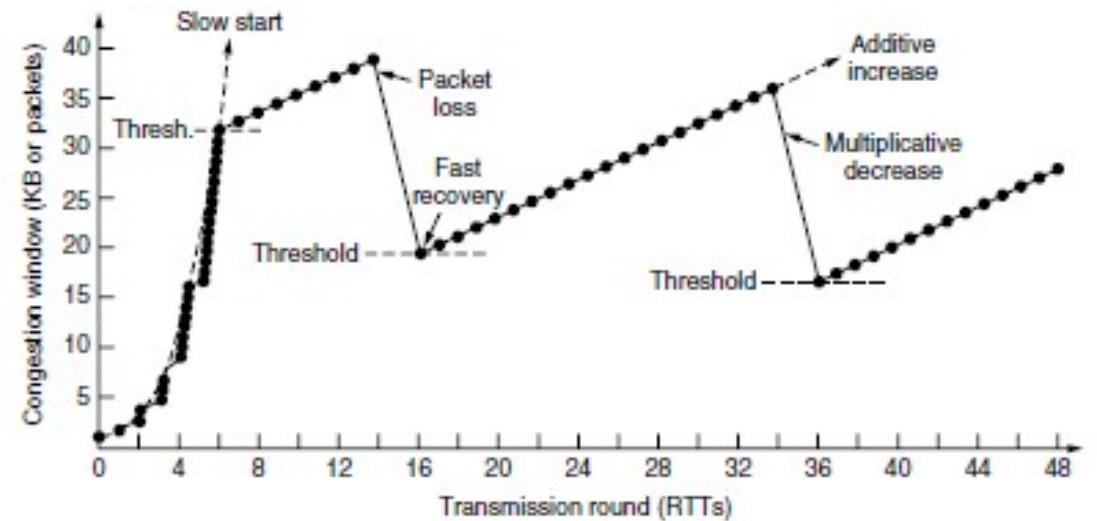
TCP Congestion Control – Congestion Detection

- A TCP sender perceives that there is congestion on the path between itself and the destination in two ways by checking ***loss events***.
- A ***loss event*** happens **when either a timeout is reached or 3 duplicate/wrong ACKs are received**.
 - Both events mean that a previous packet **is not arrived at the destination** probably because of network devices overflow.
- If **loss events do not occur**, i.e., ACKs of segments arrive as expected, TCP will assume that network is not congested, so the **congestion window can be increased**.
- If, on the other hand, **loss events occur**, the **congestion window must be decreased**.

Transport Layer

TCP Congestion Control – Rate Adjustment

- TCP rate adjustment is then performed through **bandwidth probing**: rate is increased as long as ACKs arrive correctly (probing the network), when congestion is detected (loss events) the rate is decreased. This process is continuously repeated.
- TCP uses **Jacobson congestion-control algorithm** [Jacobson 1988] to regulate the rate, it includes 3 phases:
 1. **Slow start**: start from 1 MSS/RRT increase rate exponentially.
 2. **Additive Increase** (or congestion avoidance): increase rate linearly.
 3. **Fast Recovery** (optional): halves the rate instead of slow-starting again and proceed with additive increment.



Typical sawtooth behavior of the Jacobson algorithm